



CHAPTER 11

Inheritance & More On OOPS



Inheritance is a way of creating a new class from an existing class.

Syntax:

Inheritance helps in code reusability and reduces redundancy!

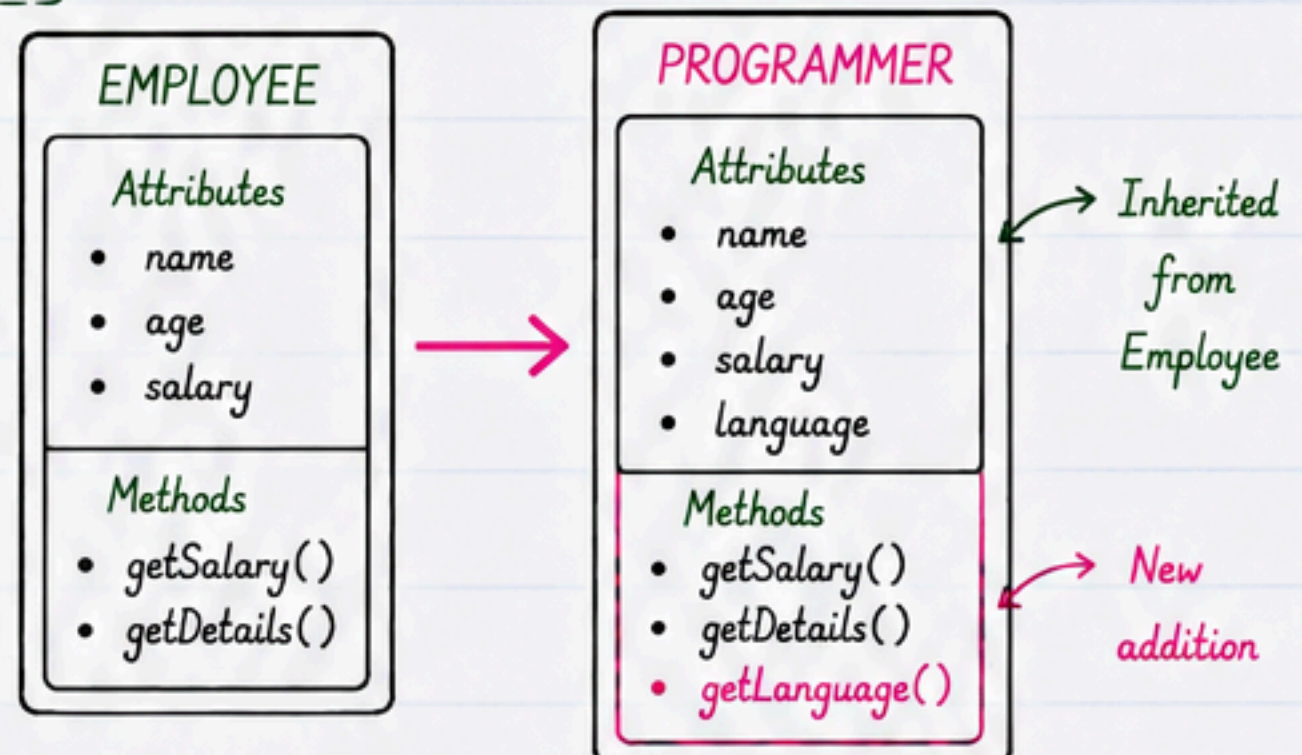


```
class Employee:           # Base class
    # Code

class Programmer(Employee): # Derived or child class
    # Code
```

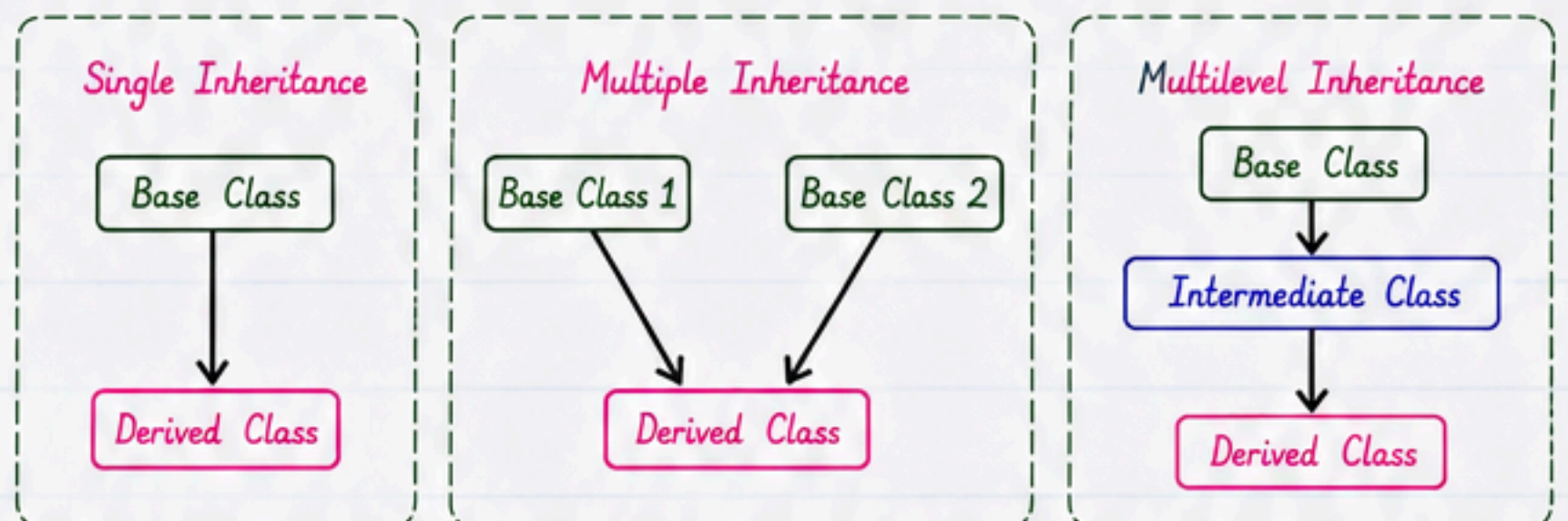
★ We can use the method and attributes of 'Employee' in 'Programmer' object.

★ Also, we can overwrite or add new attributes and methods in 'Programmer' class.



Types Of Inheritance

- 1 Single inheritance
- 2 Multiple inheritance
- 3 Multilevel inheritance

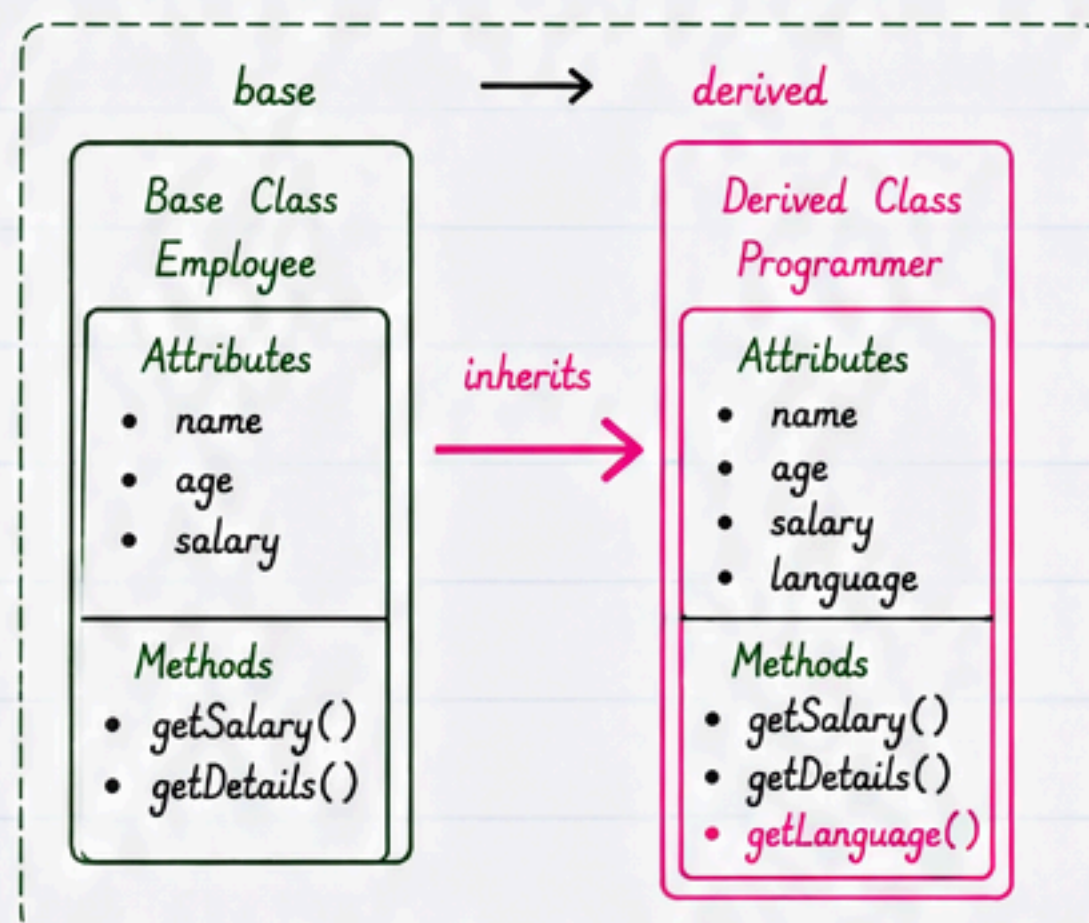


Single Inheritance

Single inheritance occurs when child class inherits only a single parent class.



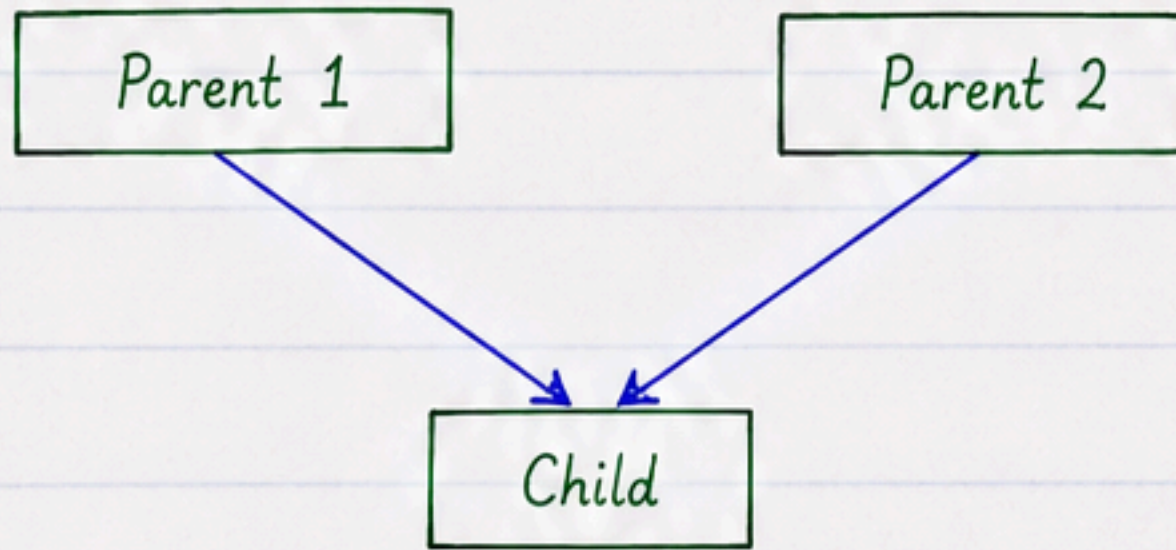
It promotes code reusability and establishes a clear parent-child relationship.



• Multiple Inheritance

Multiple Inheritance occurs when the child class inherits from more than one parent classes.

Child class inherits features from multiple parent classes.



Example:

```
class A:
    def feature1(self):
        print("Feature of class A")

class B:
    def feature2(self):
        print("Feature of class B")

class C(A, B): # C inherits from A and B
    def feature3(self):
        print("Feature of class C")
```

Child class C gets features from both A and B.



• Multilevel Inheritance

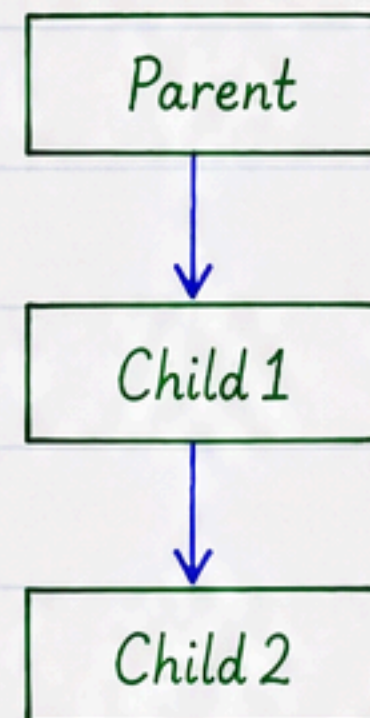
When a child class becomes a parent for another child class.

Example:

```
class A:
    def feature1(self):
        print("Feature of class A")

class B(A): # B inherits from A
    def feature2(self):
        print("Feature of class B")

class C(B): # C inherits from B
    def feature3(self):
        print("Feature of class C")
```



Inheritance proceeds in a chain or level by level.

Explanation:

- Child1 inherits features from Parent.
- Child2 inherits features from Child1, and indirectly from Parent.
- So, Child2 has access to all features of Parent and Child1.



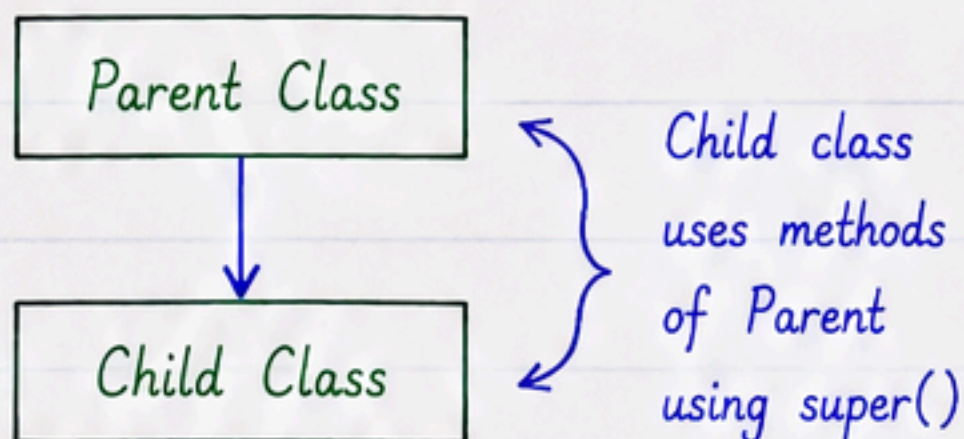
• super() Method

super() method is used to access the methods of a super class in the derived class.

```
super().__init__()
```

`__init__()` Calls constructor of the base class

super() helps the child class use the features of its parent.



• Class Method

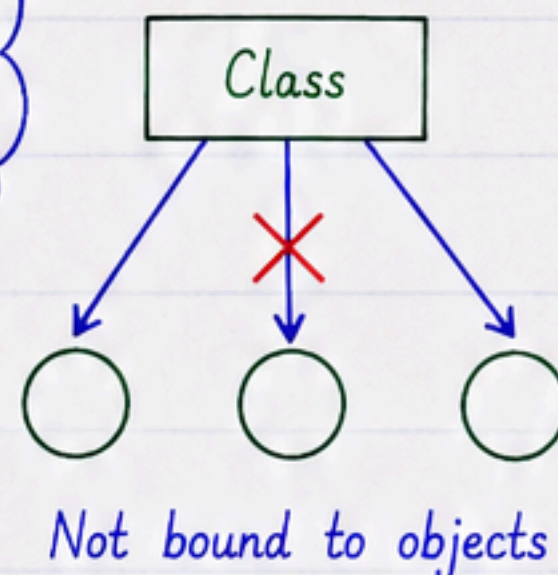
A class method is a method which is bound to the class and not the object of the class.

@classmethod decorator is used to create a class method.

Syntax:

```
@classmethod
def(cls, p1, p2):
```

Bound to the class, not to the object.



• @property Decorators

Consider the following class:

```
class Employee:
    @property
    def name(self):
        return self.ename
```



@property allows us to access methods like attributes.

If `e = Employee()` is an object of class employee, we can print(`e.name`) to print the ename by internally calling `name()` function.

• @.getters and @.setters

The method name with '@property' decorator is called getter method.

We can define a function + @name.setter decorator like below:

```
@name.setter
def name(self, value):
    self.ename = value
```

Getter vs Setter

- @property → Getter (to read value)
- @name.setter → Setter (to set value)



Operator Overloading In Python

- Operators in Python can be overloaded using **dunder methods**.
- These methods are called when a given operator is used on the objects.

Operator overloading allows custom objects to behave like built-in types with operators.



★ Operators in Python can be overloaded using the following methods:

$p1 + p2$	$\longrightarrow$	$p1.__add__(p2)$
$p1 - p2$	$\longrightarrow$	$p1.__sub__(p2)$
$p1 * p2$	$\longrightarrow$	$p1.__mul__(p2)$
$p1 / p2$	$\longrightarrow$	$p1.__truediv__(p2)$
$p1 // p2$	$\longrightarrow$	$p1.__floordiv__(p2)$

Example:

```
c = a + b
    calls a.__add__(b)
c = a - b
    calls a.__sub__(b)
c = a * b
    calls a.__mul__(b)
c = a / b
    calls a.__truediv__(b)
c = a // b
    calls a.__floordiv__(b)
```



Other dunder/magic methods in Python:

__str__()

used to set what gets displayed
upon calling str(obj)

Example:

```
class Book:
    def __init__(self, title):
        self.title = title
    def __str__(self):
        return f"Book: {self.title}"

b = Book("Python Basics")
print(str(b)) # Output: Book: Python Basics
```

__len__()

used to set what gets displayed
upon calling len(obj)

Example:

```
class MyList:
    def __init__(self, items):
        self.items = items
    def __len__(self):
        return len(self.items)

m = MyList([10, 20, 30, 40])
print(len(m)) # Output: 4
```



Key Points:

- Dunder (double underscore) methods allow customization of object behavior.
- Operator overloading makes user-defined objects more intuitive to use.
- These methods are automatically called by Python when the corresponding operator or function is used.

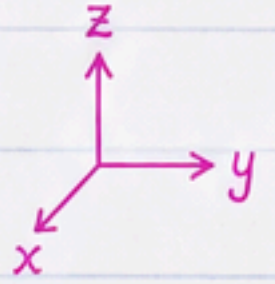
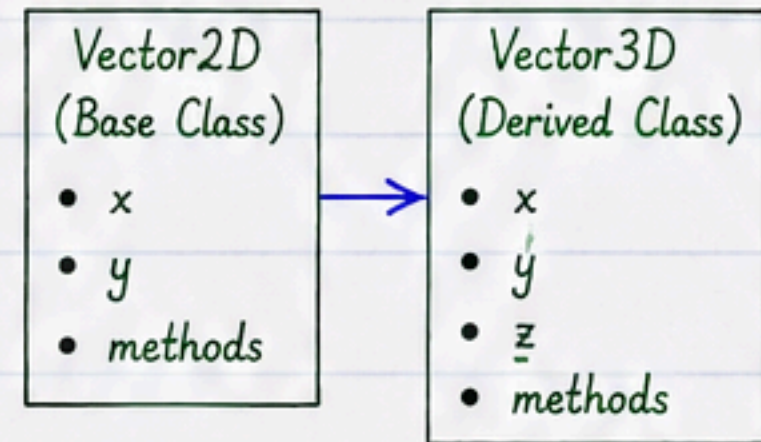




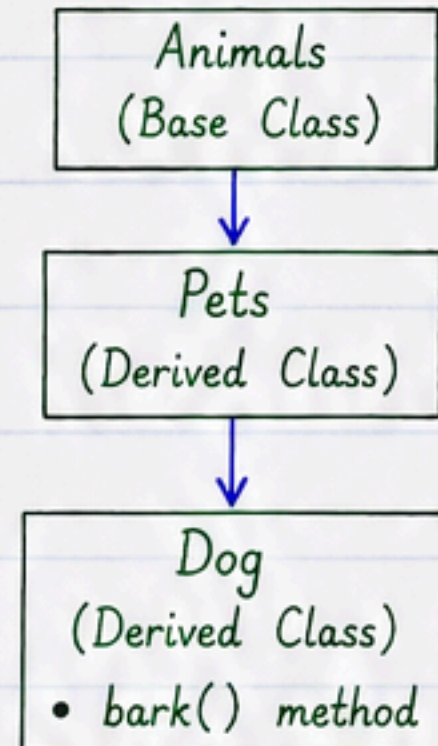
Practice Set



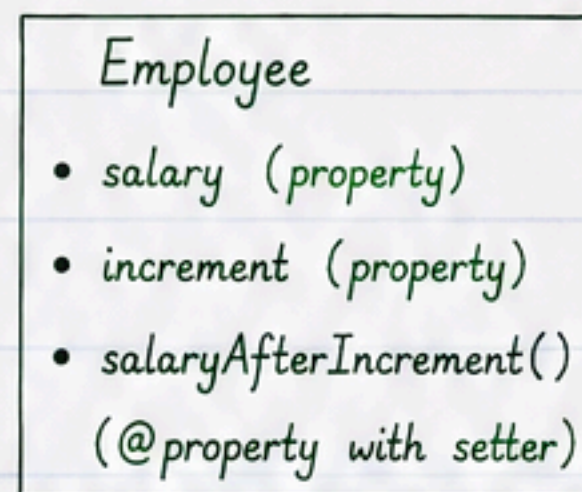
1. Create a class (2-D vector) and use it to create another class representing a 3-D vector.



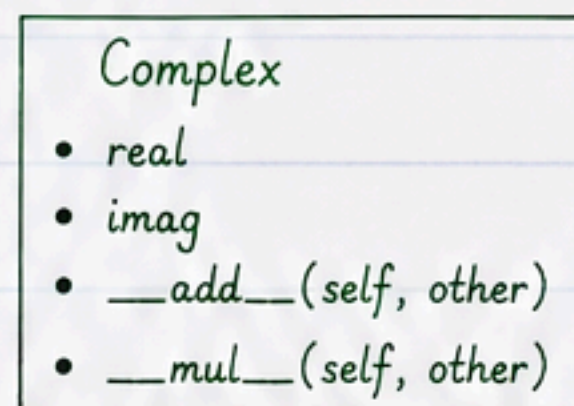
2. Create a class 'Pets' from a class 'Animals' and further create a class 'Dog' from 'Pets'. Add a method 'bark' to class 'Dog'.



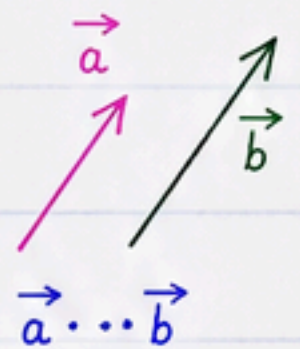
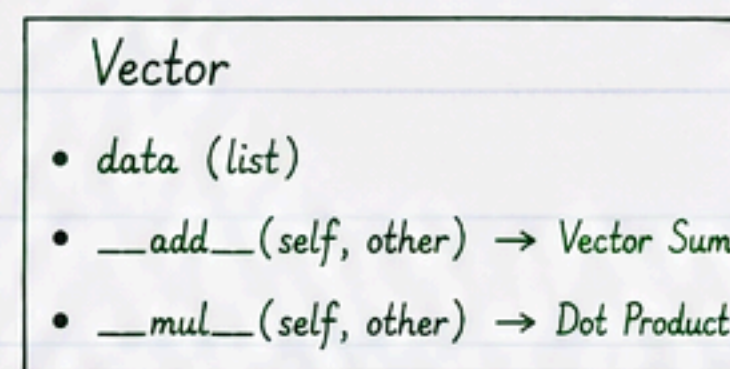
3. Create a class 'Employee' and add salary and increment properties to it. Write a method 'salaryAfterIncrement' method with a @property decorator with a setter which changes the value of increment based on the salary.



4. Write a class 'Complex' to represent complex numbers, along with overloaded operators '+' and '*' which adds and multiplies them.



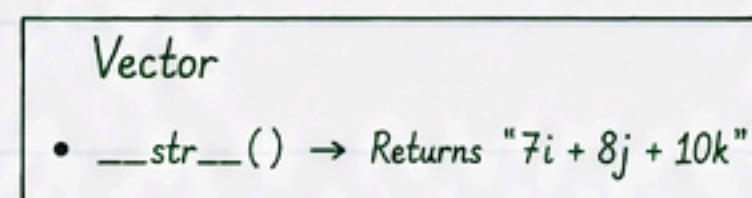
5. Write a class vector representing a vector of n dimensions. Overload the + and * operator which calculates the sum and the dot(.) product of them.



6. Write __str__() method to print the vector as follows:

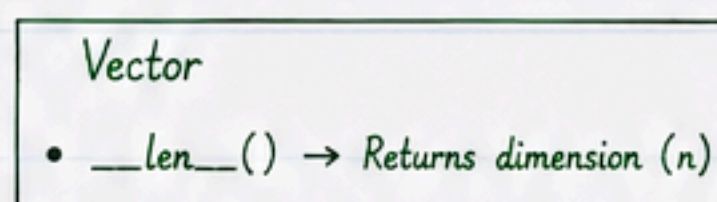
$$7i + 8j + 10k$$

Assume vector of dimension 3 for this problem.



$$7i + 8j + 10k$$

7. Override the __len__() method on vector of problem 5 to display the dimension of the vector.



$$\text{len}(v) = n$$

